

```

javascript
let p = ["P1", "P2", "P3"]; //ARRAY PROCESSI
let at = [3, 2, 5]; //ARRAY TEMPO DI ARRIVO
let bt = [2, 4, 2]; //ARRAY TEMPO DI BURST
let rbt = []; //ARRAY TEMPO DI BURST RIMANENTE
let wt = []; //ARRAY TEMPO DI ATTESA
let op = []; //ARRAY ID o INDICI PROCESSI ORDINATO
let t; //TEMPO DEL PROCESSORE
let idfirst; //ID o INDICE DEL PROCESSO DA ESEGUIRE
let tottime; //TEMPO DI TURNAROUND TOTALE

```

```

/* FUNZIONE RESET

```

- 1) La tabella dei processi viene sostituita con una tabella vuota
- 2) Il tempo di evoluzione t viene inizializzato a 0
- 3) Il contenuto dei div relativi ai messaggi di output viene cancellato

```

*/

```

```

function reset(){
    let tableEl = document.getElementById("idTable");
    let oldTBodyEl = tableEl.getElementsByTagName('tbody')[0];
    let newTBodyEl = document.createElement('tbody');
    t = 0;

    tableEl.replaceChild(newTBodyEl, oldTBodyEl);

    document.getElementById("idTime").innerHTML = "";
    document.getElementById("idStatistics").innerHTML = "";
}

```

```

/* FUNZIONE START

```

- 1) si inizializzano le variabili relative al tempo del processore t e tottime
- 2) si inizializzano gli array rbt, wt, op
- 3) si mostra la variabile tempo t
- 4) si inseriscono nel corpo della tabella i dati dei processi (nome, tempo di arrivo, tempo di burst)
- 5) si ordina (Selection sort) l'array op con l'indice dei processi in funzione dell'algoritmo di scheduling scelto  
(FCFS: ordine in base al tempo di arrivo  $at[i] > at[j]$ )
- 6) si determina il primo processo da eseguire aggiornando idfirst

```

*/

```

```

function start(){

```

```

    let i;
    //1) si inizializzano le variabili relative al
    //tempo del processore t e tottime

```

```
t = 0;
tottime = 0;
```

```
//2) si inizializzano gli array rbt, wt, op
for (i = 0; i < p.length; i++){
    rbt[i] = bt[i];
    wt[i] = 0;
    op[i] = i;
}
```

```
//3) si mostra la variabile tempo t
document.getElementById("idTime").innerHTML = "Tempo: " + t;
```

```
//4) si inseriscono nel corpo della tabella i dati dei processi (nome, tempo di arrivo, tempo di burst)
let tableEl = document.getElementById("idTable");
```

```
let oldTBodyEl = tableEl.getElementsByTagName('tbody')[0];
let newTBodyEl = document.createElement('tbody');
for(i=0; i<p.length; i++) {
    const trEl = newTBodyEl.insertRow();
    let tdEl = trEl.insertCell();
    tdEl.appendChild(document.createTextNode(p[i]));
    tdEl = trEl.insertCell();
    tdEl.appendChild(document.createTextNode(at[i]));
    tdEl = trEl.insertCell();
    tdEl.appendChild(document.createTextNode(bt[i]));
    tdEl = trEl.insertCell();
```

```
tdEl.id = "idP" + i;
tdEl.appendChild(document.createTextNode(rbt[i]));
```

```
}
tableEl.replaceChild(newTBodyEl, oldTBodyEl);
```

```
//5) si ordina (Selection sort) l'array op con l'indice dei processi in funzione dell'algoritmo di scheduling scelto
```

```
 //(FCFS: ordine in base al tempo di arrivo at[i] > at[j])
```

```
let j, temp;
console.log(op);
for (i=0; i<p.length - 1; i++)
```

```

for (j=i+1;j<p.length;j++){
  if (at[i] > at[j]) {
    temp=op[i];
    op[i]=op[j];
    op[j]=temp;
  }
}
console.log("ordine processi: " + op);

```

//6) si determina il primo processo da eseguire aggiornando idfirst  
idfirst = op[0];

```

function findWaitingTime() { let serviceTime = Array(p.length).fill(0); wt[0] = 0; // Il primo
processo non ha tempo di attesa // Calcola il tempo di attesa for (let i = 1; i < p.length; i++)
{ serviceTime[i] = serviceTime[i - 1] + bt[op[i - 1]]; wt[op[i]] = serviceTime[i] - at[op[i]];
if (wt[op[i]] < 0) wt[op[i]] = 0; // Se il processo è arrivato dopo, non può avere tempo di
attesa negativo } console.log("Tempi di attesa: " + wt); }
}

```

```

function step(){
}

```

```

html
<!DOCTYPE html>
<html>
<head>
  <title>Simulazione Scheduler</title>
  <style>
    table { border-collapse: collapse; width: 100%; }
    th, td { border: 1px solid black; text-align: center; padding: 8px; }
    th { background-color: #f2f2f2; }
  </style>
</head>
<body>
  <h1>Simulazione Scheduler</h1>
  <button onclick="reset()">Reset</button>
  <button onclick="start()">Start</button>
  <button onclick="step()">Step</button>
  <div>
    <h2>Tabella dei Processi</h2>

```

```
        <table id="processTable"></table>
    </div>
    <div id="output"></div>
</body>
</html>
```